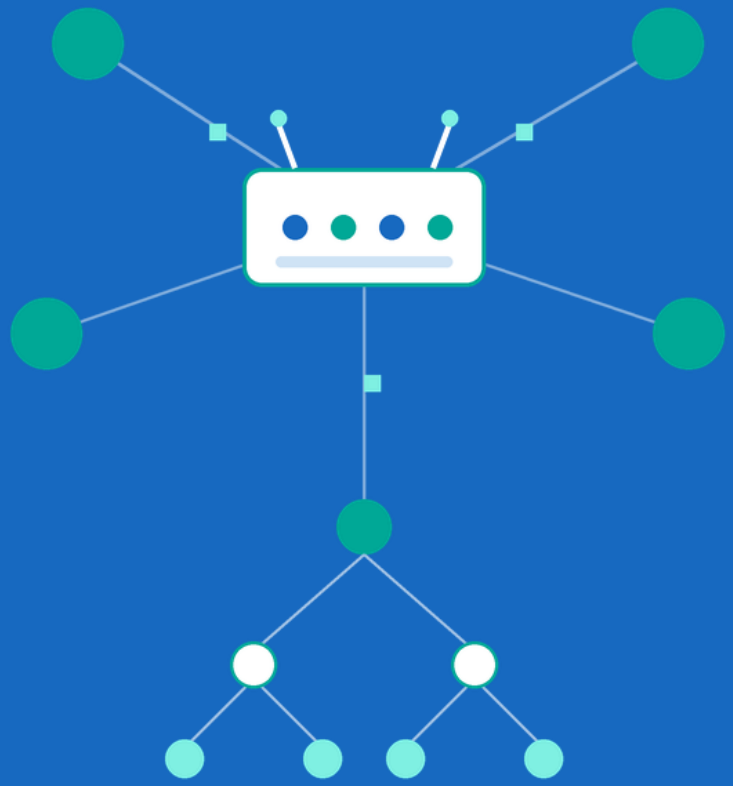
IP Router Cache Lookup

CS 212 — Final Project



Outline

- Introduction
- Project Analysis
- System Architecture
- Project Implementation
- Results & Benchmarks
- Conclusion

Introduction

- Routers forward each packet based on its destination IP address
- They must find the MOST DIRECT matching route - Longest Prefix Match (LPM)
- At scale this must be extremely fast → what motivates a cache implementation
- Every packet needs its own routing decision, and a core router makes millions of those per second against a routing table that can hold close to a million routes.

Project Analysis

Real traffic most of the time repeats, having a handful of destinations dominate. We can cache recent lookups and skip the expensive routing table search.

Approach	Lookup Speed	Memory	Notes
Linear scan of routes	O(n)	Low	Doesn't scale to large tables
Trie (LPM)	O(W) bits	Higher	Standard router approach
Trie + Lookaside Cache	**~O(1) on cache hit**	Higher	Fast for repeated traffic

IP Router Cache Lookup — System Architecture

Simulator / main.cpp

- generates traffic workload
- injects table route changes

lookup(address) · add_route() · remove_route()

RouteCache

- unordered_map (custom IpHash)
- check cache FIRST → HIT returns cached port → skip trie
- MISS → RoutingTable lookup store → result in cache
- invalidate(prefix, len) on table route change → drop outdatet/no longer true entries
- metrics collected: hits / total

hits / total

Metrics / Report

- hit rate
- total lookups
- cached vs uncached time
- speedup

result: (port) → fill cache

on MISS:
lookup(address)

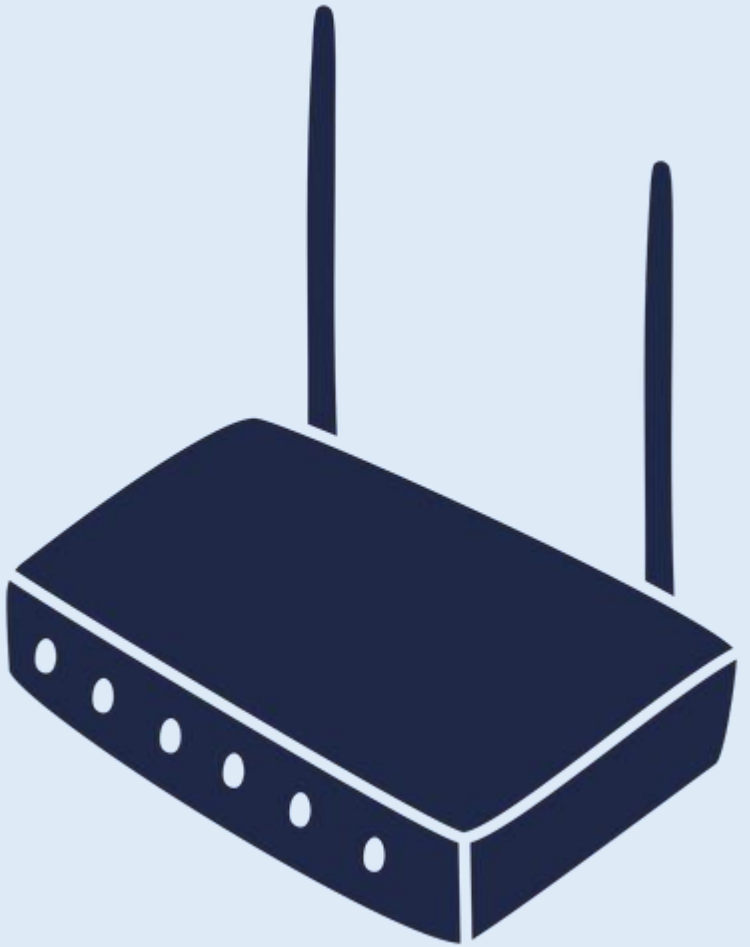
RoutingTable

- add_route(prefix, port) / remove_route(prefix)
- lookup(addr) → forwards to the trie
- holds a Trie (composition)

LPM walk
(bit by bit)

Trie (binary, per-bit)

- Longest Prefix Match (deepest marked node)
- insert / remove / LPM
- templated for diff address types: uint32_t = IPv4, __int128 = IPv6



Project Implementation

- 1. Routing lookup table (Trie / long-form table)
- 2. Longest Prefix Match algorithm
- 3. Lookaside cache to skip heavy Trie lookups
- 4. Cache consistency when routes change / drop
- 5. Benchmark workloads — hit rate & lookup speed

Results & Benchmarks

- Cache hit rate: 90.0054 %
- Avg lookup time WITHOUT cache: 19.3029 ms
- Avg lookup time WITH cache: 15.0766 ms
- Speedup: 1.28032x

Conclusion

- ***Built:*** A complete IP-router lookup simulator: LPM trie → lookaside cache → consistency
- **Takeaway:** caching exploits traffic locality to skip the expensive Longest Prefix Match - turning repeated lookups into optimized O(1) hits
- ***Learned:***
  - *What is* Longest Prefix Match and why using Trie is the right structure
  - Cache trades speed for a consistency problem → route changes make entries stale, so we need to invalidate them
  - A custom hash for 128-bit addresses for IPv6 was required. C++'s std::hash returns a 64-bit size_t and has no built-in hash for 128-bit types, so we combine two 64-bit halves to one.

Thank you for your attention!